Once a transaction acquires the `PESSIMISTIC_FORCE_INCREMENT` lock and increments the entity version, no other transaction can acquire a `PESSIMISTIC_FORCE_INCREMENT` lock because the second select statement is blocked until the first transaction releases the row-level physical lock.

The following example aims to demonstrate how two concurrent transactions can be coordinated through a common entity `PESSIMISTIC_FORCE_INCREMENT` lock acquisition.

```
doInJPA(entityManager -> {
    Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.PESSIMISTIC_FORCE_INCREMENT);

    executeAsync(() -> doInJPA(_entityManager -> {
        startLatch.countDown();
        Repository _repository = _entityManager.find(Repository.class, 1L,
            LockModeType.PESSIMISTIC_FORCE_INCREMENT);

        Commit _commit = new Commit(_repository);
        _commit.getChanges().add(new Change("Intro.md", "0a1,2..."));
        _entityManager.persist(_commit);
        _entityManager.flush();
        endLatch.countDown();
    }));
    awaitOnLatch(startLatch);
    LOGGER.info("Sleep for 500ms to delay the other transaction");
    sleep(500);

    Commit commit = new Commit(repository);
    commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
    commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));

    entityManager.persist(commit);
});
endLatch.await();
```

The `awaitOnLatch` and `sleep` method utilities have the role of converting the `InterruptedException` into a `RuntimeException` which, unlike checked exceptions, can be propagated throughout a lambda expression without having to add unnecessary try/catch clauses.

When executing the test case above, Hibernate generates the following statements:

```sql
-- Alice selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

-- Alice increments the Repository version upon fetching the Repository entity
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Bob tries to select the Repository entity, but his select is blocked
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

-- Alice waits 500 ms to delay Bob's lock acquisition request

-- Alice adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,5...', 'FrontMatter.md')

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '17c17...', 'HibernateIntro.md')

-- Bob's increments the Repository version upon fetching the Repository entity
UPDATE repository SET version = 2 WHERE id = 1 AND version = 1

-- Bob adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 3)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (3, '0a1,2...', 'Intro.md')
```

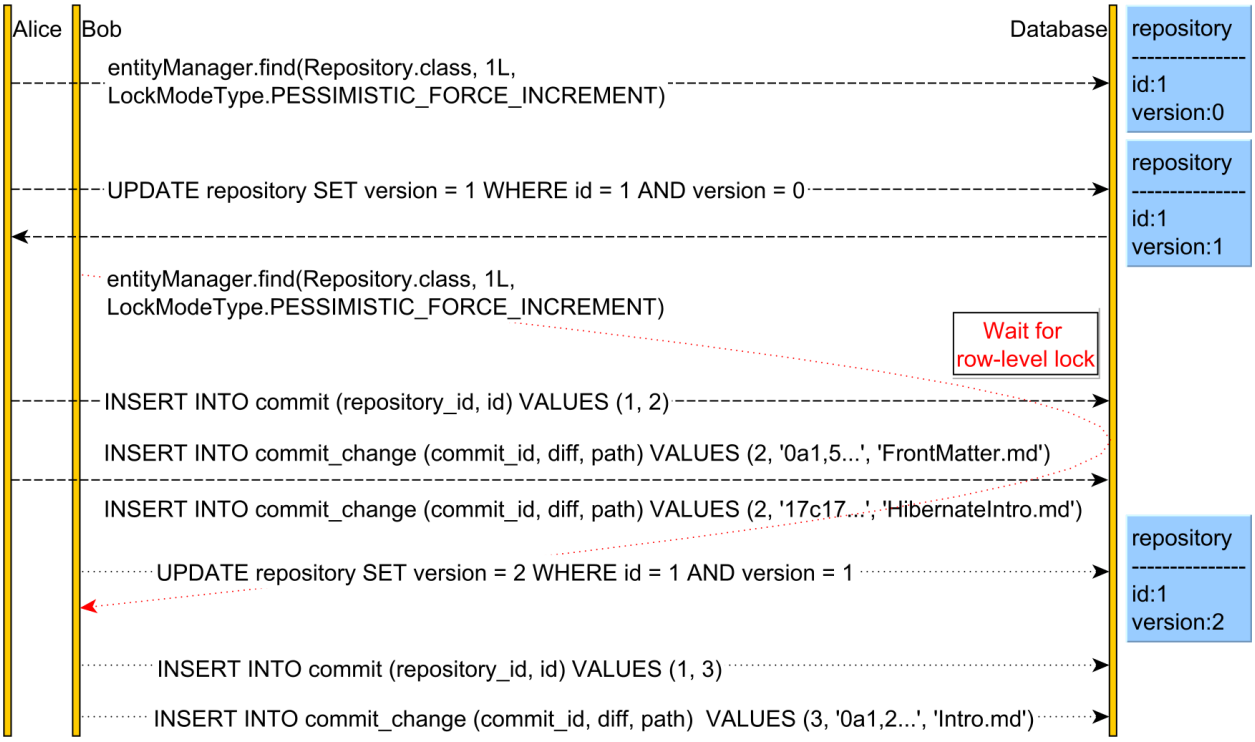

Figure 16.13: LockModeType.PESSIMISTIC_FORCE_INCREMENT

The flow can be explained as follows:

- Alice fetches the `Repository` entity while also acquiring a row-level lock and incrementing the entity version.
- Alice's thread is suspended by the JVM thread scheduler, so Bob gets the chance to fetch the `Repository` entity using the `PESSIMISTIC_FORCE_INCREMENT` lock strategy.
- Because it uses a `FOR UPDATE` clause, Bob `Repository` entity select statement is blocked by Alice's row-level lock.
- Alice thread is resumed, and she waits for 500 ms to delay Bob's lock acquisition request.
- Alice adds a new `Commit` entity, and her transaction is committed. Therefore, the `Repository` entity row-level lock is released.
- Bob can resume his select statement, and he acquires a row-level lock on the `Repository` entity.
- The `Repository` entity version is incremented by the `PESSIMISTIC_FORCE_INCREMENT` lock strategy.
- Bob manages to add his `Commit` entity and commits his transaction.

Once the row-level lock is acquired, the entity version update is guaranteed to succeed, therefore, reducing the likelihood of getting an `OptimisticLockingException`.

III JOOQ

17. Why jOOQ matters

When working with a relational database, it all boils down to SQL statements.

As previously explained, Hibernate entity queries are suitable for read-write logical transactions. For reporting, analytics or ETL (Extract, Transform, Load) native SQL queries are the best choice since they can take advantage of database-specific features like window functions or Common Table Expressions. Even for CRUD operations, there might be times when a database-specific syntax is more suitable like it's the case for the upsert SQL operation[1].

While Hibernate does a very good job to automate the vast majority of statements, it is unlikely that you can rely on Hibernate alone for every business use case. Therefore, native queries are a necessity for most enterprise applications.

As demonstrated in the Native query DTO projection section, both JPA and Hibernate provide a way to execute native SQL statements. Being able to execute any SQL statement is great, but, unfortunately, the JPA approach is limited to static statements only. To build native SQL statement dynamically, JPA and Hibernate are no longer enough.

17.1 How jOOQ works

JOOQ is a query builder framework that allows you generate a great variety of database-specific statements using a Java API. The `DSLContext` is the starting point to building any SQL statement, and it requires two things:

- a reference to a JDBC `Connection`
- a database dialect so that it can translate the Java API query representation into a database-specific SQL query

For instance, when using PostgreSQL 9.5, the `DSLContext` can be constructed as follows:

```
DSLContext sql = DSL.using(connection, SQLDialect.POSTGRES_9_5);
```

17.2 DML statements

With the `DSLContext` in place, it's time to show some simple DML statements like insert, update, delete, as well as a trivial select query. What's worth noticing is that the Java API syntax is almost identical to its SQL counterpart, so most jOOQ queries are self-describing.

[1] https://en.wikipedia.org/wiki/Merge_(SQL)